

Checkpoint 5

Due: April 13th, by 11:59pm

OVERVIEW

You have now reached the point where you have a fully functional game. Before continuing, it is critical that you take this opportunity to review your work in all previous stages. Ensure that all requirements described in those stages are met. This includes ensuring each of the following:

- That all aspects of the graphics are complete, including flicker/tear-free double buffered animation
- That the rendering is efficient and the animation is smooth, at or approaching 70fps
- That all aspects of the music and sound effects are complete
- That the core 1-player game play and functionality are finished, from the user's perspective
- That the title screen which responds to keyboard inputs has been implemented
- That the game specification document from stage 1 is in-sync with the game itself (update the spec. and/or the game as necessary)
- That the code is well designed and clearly written, and conforms to all the code and documentation standards described in earlier checkpoints

This might look like the end of the line, but there are still improvements that can be made that will improve game play in the game. This checkpoint will focus on removing your game's reliance on the operating system by replacing almost all system calls in your code with your own custom code. This checkpoint will be broken into three different sections each of which will focus on eliminating one portion of your games reliance on the operating system.

At the completion of this checkpoint the only system call remaining must be the ***Super*** system call. The *Super* system call will be removed in one of the bonus checkpoints if you decide to attempt them. This implies that the code does not issue any system calls other than *Super*, nor does it rely on any operating system variables/tables. The TOS font table and key table are examples of the O/S variables/tables, and are therefore strictly disallowed (unless you've *ripped* your own copies).

This also implies that any library functions which cause TOS system calls to be invoked indirectly are also forbidden. In particular, you may not use I/O functions such as `printf` or `getchar`. You must also avoid `malloc` and `free`, since they may occasionally need to interact with the operating system. You must also avoid `time` (e.g. `time(NULL)`) as this relies on the

system time which is a TOS system call. The implementations of `srand`¹ and `rand` do not invoke the operating system, so their usage is safe. Similarly, string manipulation functions such as `strcpy` and `strcmp` are safe. If in doubt about a particular function, seek guidance from your professor before continuing to use it.

REPLACING VIDEO RELATED SYSTEM CALLS

Background: Atari ST Video Hardware Summary (Monochrome)

As discussed previously, the frame buffer (FB) is a 16,000 word region of RAM. It holds the bitmap for the video image to display on the monitor's screen. The ST's video hardware repeatedly indexes through FB, word-by-word. The SHIFTER chip serializes each successive word to produce a video intensity signal, which is output on a pin of the video port.

Actually, the SHIFTER does not index memory directly. Another chip, called the MMU (Memory Management Unit), is responsible for addressing RAM. Each indexed word is placed on the data bus by RAM, and is then latched by the SHIFTER.

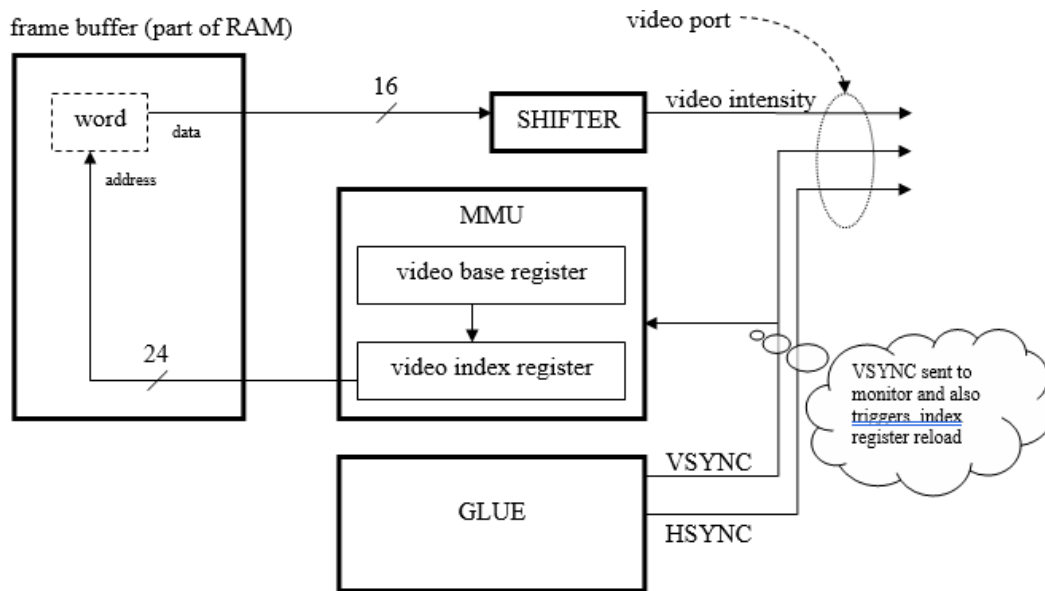
Yet another chip, the GLUE, is responsible for generating the horizontal and vertical synchronization signals (HSYNC and VSYNC) for the monitor. They specify the timing of horizontal and vertical scans. These two signals are output on the video port along with the video intensity signal. The entire screen is refreshed at a rate of 70 Hz.

Obviously, the MMU maintains a "video index register" to keep track of the current FB word to address. This 24-bit counter is periodically incremented by 2, so that it steps through the addresses of FB memory. The MMU also maintains a "video base register". This holds a pointer to the start address of the FB. At the end of each pass through the FB, the index register is reloaded with the contents of the base register in preparation for the next screen refresh.

Important: it is by altering the contents of the video base register that the frame buffer start address can be changed. This is how page flipping is accomplished when performing double buffered graphics. **Note carefully:** when a new value is written into the base register, the value is **not** loaded into the video index register until the next VSYNC! So, the page flip does not occur immediately. It occurs some time within the next 1/70th of a second.

¹As long as you are not doing an invocation of `srand` with `time` as the parameter.

The diagram below summarizes this arrangement:



One final comment for the curious: on the ST, the MMU handles all RAM accesses. If the 68000 places a valid RAM address on the address bus, the MMU forwards this address along to RAM. If the SHIFTER needs another word of data, again, the MMU addresses RAM at the next video index. The latter happens repeatedly, without the CPU's involvement. If the CPU and the SHIFTER both require memory to be accessed at roughly the same time, the MMU alternates memory access cycles between them.

Replace Physbase & Setscreen

In this section, you must remove the `Physbase` and `Setscreen` system calls, and replace them with your own implementations. You will create your own versions of the `Physbase` and `Setscreen` functions, called: `get_video_base()` and `set_video_base` respectively. These two functions will be placed in the *raster* module.

Replace Physbase

Write your own routine for retrieving the current frame buffer start address:

```
UINT16 *get_video_base();
```

Replace each call to `Physbase` with a call to `get_video_base`.

If you consult a memory map, you will discover that there is a 16-bit video base register (spread over two 8-bit addresses) which holds the frame buffer start address. These two bytes represent the high and middle bytes of the 24-bit video base. The low 8 bits are always zero, and are not stored².

Replace Setscreen

Once `get_video_base` is working, write your own routine for changing the frame buffer start address:

```
void set_video_base(UINT16 *);
```

Replace each call to `Setscreen` with a call to `set_video_base`.

Implementing these two functions in C does not require much code. However, to be fully correct, the latter function must really be implemented in 68000 assembly language using the `movep` instruction. This is so the two bytes of the 16-bit video base register are updated at the same time. Not doing so can trigger a bug if VSYNC happens midway through the `set_video_base` algorithm's execution, at the point when the two 8-bit portions of the video base register are inconsistent³.

CUSTOM ISRs

In this section you will replace two of TOS's ISRs with two of your own. Specifically, you will write custom VBL (vertical blank) and IKBD ISRs. We will refer to the development of each of these ISRs as two sub-sections. Budget about 1-2 days to complete the VLB ISR sub-section⁴, and 3-4 days for the IKBD ISR sub-section.

From the user's perspective, the impact of these changes on the game's functionality should be almost non-existent⁵. However, your program's reliance on the operating system will be eliminated almost completely!

General ISR Requirements (Applies to Both VBL and IKBD ISR Sub-Stages)

As you know, an ISR must be implemented in assembly language. However, it is acceptable to delegate one or more subtasks to subroutines written in C. Of course, this may not be necessary for very simple subtasks.

²This is why frame buffer addresses on the ST must be 256-byte aligned.

³This is from the fact that it would require two different `move` operations to change the video base register and an interrupt can be signalled before first has finished being executed but before the second, which means the VBL interrupt would occur with a partial address in the video base register

⁴If all previous stages have been designed and implemented well, the VBL ISR should actually be very quick.

⁵One exception is that the text cursor maintained by TOS will cease to blink after implementing the VBL ISR. This is because the plotting of the text cursor is performed in the TOS VBL ISR.

ISRs *must always* be as *fast* as possible! The idea is to do the minimum amount of work and then return quickly to the interrupted task. Remember that other, lower priority (but still important) ISRs may be delayed due to the execution of your ISR.

Time-consuming operations must be left for the main program to perform. Typically, an ISR will simply update some global data (if necessary) and then will somehow notify the main program (e.g. by setting a flag or enqueueing in a buffer) that some extra work needs to be performed.

Ideally, your game supports the ability to quit back to TOS. When an ISR is installed, the previous vector must be saved so that it can be restored on exit.

Requirement: VBL ISR

The goal of this sub-stage is to implement a VBL ISR for your game. This will enable you to reduce the amount of clock polling performed by the main game loop.

You must replace the TOS VBL ISR with your own. It must:

⁶

- Time the page flipping for double buffered graphics
- Time the playing of music
- Time any other synchronous/conditional events needed by your game
- Time the update of the game model based on any asynchronous events since the last vertical blank

Guidance on how to accomplish each of the above are given in the next three sections.

Vertical Blank Notes

Since your game will have a maximum frame rate of 70 Hz, there is no point in generating synchronous events using a faster timer. Put another way, why update your model more frequently than you can render views of the model? Regardless, some games may require a higher frequency, e.g. for measuring a user's reaction time. In this case, you can use one of the ST's programmable hardware timers. However, such issues will *not* be dealt with in the IKBD ISR sub-stage and will not be discussed here. Seek additional guidance from your instructor as necessary.

⁶Since your game will have a maximum frame rate of 70 Hz, there is no point in generating synchronous events using a faster timer. Put another way, why update your model more frequently than you can render views of the model? Regardless, some games may require a higher frequency, e.g. for measuring a user's reaction time. In this case, you can use one of the ST's programmable hardware timers. However, such issues will *not* be dealt with in the IKBD ISR sub-stage and will not be discussed here. Seek additional guidance from your instructor as necessary.

Your main game loop must **not** perform any clock polling. This includes no longer consulting the TOS vertical blank counter (the longword at address \$462, sometimes referred to as *vbclock*)⁷.

VBL ISR & vbclock

Your custom VBL ISR **MUST NOT** simply increment the vbclock. This will result in severe mark reduction for the VBL ISR portion of the grade.

Guidance: Timing of Page Flipping (VBL ISR)

One way of achieving flicker and tear free animation is to perform all screen drawing inside the VBL ISR, i.e. between screen refreshes, when the frame buffer isn't being scanned. However, this only works for very short plotting operations. This technique will not usually work for games such as yours because the image rendering takes too long. If your code is still updating the frame buffer when the next screen refresh begins, then image distortion will be seen.

Another common (but incorrect) approach is to render the next frame of animation in the main routine, but to then flip the buffers inside the ISR. The idea (again, incorrect) is that page flips will be synchronized with screen refreshes. This will not work because the video index register is reloaded from the video base register at the same time as the VBL IRQ is asserted. By the time your ISR is invoked, it is too late to update the video base.

The correct technique for your game is as follows: 70 times per second, the VBL ISR requests that the next frame of animation be rendered, e.g. by setting a global *render request* flag. When the main game loop sees that "render request" has been set, it calls the master render routine to render the next frame of animation into the back buffer. Then, a page flip is scheduled by calling `set_video_base`. The *render request* is then cleared. Note that the pages won't actually flip until the next vertical blank (when the video index register is reloaded with the new video base). If the main game loop is polling *render request* properly, it won't attempt to produce a subsequent frame of animation until after that flip has occurred.

In pseudo-code:

VBL ISR⁸:

```
update music (as necessary)
process synchronous/conditional events
update model based on asynchronous event requests (as necessary)
set render_request = 1
```

⁷In fact, vbclock is updated by the TOS VBL ISR. So, this clock will cease to tick once the operating system's own VBL ISR has been replaced.

⁸Note: the time elapsed since the last vertical blank, as measured in 70Hz clock ticks, will always be 1.

main game loop:

```
repeat until game over

    if asynchronous event(s) outstanding
    then
        process and update model as necessary

    if render_request
    then
        render model to back buffer
        schedule page flip by calling set_video_base
        set render request = 0
```

Guidance: Timing of Music/Sound Effects (VBL ISR)

Instead of polling the TOS vbclock in the main game loop to decide when it is time to move on to the next note, simply update the music within the VBL ISR. When it is time to move on to the next note, the ISR (or a delegate subroutine such as `update_music`) will simply update the state of the PSG. Since this is a relatively quick operation, it can be done exclusively within the ISR. There must be no song-related event processing left in the main game loop at all.

Be sure to initialize all song-related data structures and variables before installing the VBL ISR.

Note that the playing of sound effects will not necessarily be handled by the VBL ISR, unless they are in response to synchronous events. Sound effects which play in response to asynchronous events (e.g. user input) will be triggered elsewhere.

Important: each region of code which accesses the PSG is a critical section – figure out why⁹! Any main program code which attempts to access the PSG **must** do so with VBL interrupts masked.

Guidance: Timing of Other Synchronous Events (VBL ISR)

Likely, your game must process other types of synchronous events, e.g. enemy ships appearing or balls moving. Use the VBL ISR to trigger these events.

For example, if a ball is moving across the screen at constant velocity then update its position regularly, inside the VBL ISR. Do not couple the position updating with rendering. These are separate tasks. Note also that rendering time may not be constant; it may vary depending on the complexity of the image being produced. For the ball to move at a constant rate, its position must be updated regularly within the ISR, regardless of what the main program is busy doing.

⁹Hint: what could happen if the main program was in the middle of modifying PSG registers for a sound effect, due to an asynchronous user input event, and then the VBL ISR interjected and tried to play the next song tone? In particular, think about how the PSG does register selection.

IKBD ISR Sub-Stage

The goal of this sub-section is to implement an IKBD ISR for your game. It will take care of all keyboard and mouse input incoming from the IKBD's 6301 microcontroller.

Requirement: IKBD ISR (IKBD ISR)

You must replace the TOS IKBD ISR with your own, which. It must:

- handle all key make and break codes incoming from the IKBD;
- handle all mouse packets incoming from the IKBD.

Guidance on how to accomplish each of the above are given in the following sections.

Guidance: Basic Issues Related to Incoming IKBD Data (IKBD ISR)

Under TOS, the Atari ST uses vector #70 for IKBD IRQs. The IKBD ISR will be invoked once for each byte received by the IKBD 6850 ACIA, as sent by the IKBD's 6301 micro-controller. Note that it may also be invoked in other circumstances, such Rx overruns. A complete ISR must handle these eventualities.

Don't forget that the IKBD ISR will be invoked once for each key make code, once for each key break code, and three times for each mouse packet. Therefore, so that it doesn't confuse mouse data with scancode data, it must maintain a state. In other words, the ISR will be a little software finite state machine.

Important technical detail: the last step performed by the IKBD ISR must be to clear bit #6 of the "in service B" register of the MFP 68901 chip¹⁰, just before returning. Otherwise, the ISR will run a maximum of one time and will never be invoked again. The reason for this will be discussed further in lecture.

Remember that the ST has two 6850s: one is the IKBD interface and one is the MIDI port controller. They share a common vector #, and hence a common ISR. In addition, there is always the danger of spurious interrupts¹¹. Therefore, a complete ISR must check that the IKBD 6850 is actually asserting an IRQ, by checking its status register, before it attempts to service an interrupt.

For this assignment, the safest thing to do is probably to disable MIDI 6850 interrupts at the start of the program, and then to re-enable them at the end.

Important: in this assignment, you may write various routines whose execution may overlap in time (e.g. ISR code vs. interruptible non-ISR code). As discussed in class, ensure that

¹⁰This register is mapped at address \$FFFA11.

¹¹Spurious interrupts are due to hardware problems such as excessive electromagnetic interference: noise might induce a signal on an IRQ line even though no device is asserting IRQ. In a well designed system, of course, the danger of interference should have been minimized. At any rate, STEem is an emulator and probably doesn't simulate spurious interrupts. Still, your ISR should assume the possibility and deal with them if they arise.

“critical sections” are protected. This requires masking keyboard interrupts at the start of the critical section, and unmasking them at the end. To avoid overly restrictive masking, mask these interrupts at the 68901 MFP, not at the 68000.

Guidance: Handling Incoming Make and Break Codes (IKBD ISR)

Should the ISR perform input event handling for key presses? Only if the event handling logic (i.e. the required update to the game world model) is **very** simple and quick. This is discouraged, since overruns becomes very likely.

IKBD event handling should usually be left to non-ISR code. In this case, the ISR must enqueue each key press’s scan code in a buffer (a circular queue implemented as an array). As discussed in class, the main game loop must repeatedly check to see if the buffer is non-empty, and if so it must dequeue the key and process the event.

Guidance: Managing Key Auto-Repeat (IKBD ISR if necessary)

In some games, key presses are used for “discrete” input events, meaning that it is the depressing of the key which is important, not how long it is held down before being released. For example, a single shot might be fired from a ship each time the ‘f’ key is struck. The key must be re-struck in order to fire the next shot.

In other games, when a key is depressed, it signals the start of a “continuous” input event which is effect until the key is released. For example, as long as the “up arrow” key is held down, a ship might ascend. When the key is released, the ship stops ascending.

If your game takes input from the keyboard, it must decide whether input is discrete or continuous, and it must handle key make and break codes appropriately. They are both fairly straightforward to implement.

For continuous input, note that a timer ISR such as the VBL ISR will be involved. The exact approach will be game-specific. One common technique: as long as the break code for the most recent make code has not been received, regenerate the corresponding user input event at each timer ISR invocation. In other words, changes in key state are asynchronous events, but while the key is depressed a series of synchronous events are generated.

Guidance: Handling Incoming Mouse Packets (IKBD ISR)

Your IKBD ISR must deal with mouse packet bytes, even if the mouse isn’t used during game play. One reason is that mouse packet bytes should not be misinterpreted as key make or break codes if the mouse is jiggled during play. Another is that the splash screen in the final version of your game must be mouse-driven.

To process mouse input, maintain global mouse x and y coordinates. Each mouse packet will contain three separate items. The receipt of each by the 6850 will result in separate interrupts. Do **not** wait in the ISR for the arrival of all three bytes. It is guaranteed that the three items will be consecutive, i.e. a make or break from a key press will never occur between the items

of a mouse packet. The three parts of a mouse packet are the header and relative mouse movement data. The header indicates which if any mouse buttons are pressed and its value is 1111 10LR, where L means left mouse button and R means right mouse button. The relative mouse movement is the signed change in the x and y positions in pixels since the previous packet (these are commonly referred to as deltas: delta x and delta y). These deltas should be added to the global x and y coordinates. Be careful: the global coordinates will need to be integers, so your code will need to widen the deltas properly before adding.

IMPORTANT: drawing of the mouse is NOT to be done in the IKDB ISR. How this is handled will be covered in the next section.

COMPLETING THE MOUSE-DRIVEN SPLASH SCREEN

In an earlier stage you should already have completed a splash screen with a keyboard-driven main menu. You will now revise this menu so that it is mouse-driven.

Requirement: Splash Screen with Mouse-Driven Main Menu

You must provide a welcome splash screen which provides a main menu. At start-up, this allows the user to select between *1 player* and *quit* options at a minimum. Later, the splash screen will include a *2 player* option. When the game is over, the program may optionally return to this menu.

Display each menu option as a rectangular button. The user must be able to select an option by left-clicking within the corresponding rectangle.

When the splash screen is active, the program must maintain a mouse pointer plotted at an appropriate screen location, based on the user's mouse movement activity. The mouse pointer must not be visible during game play. When visible, the pointer must not flicker or tear.

The main menu **must not** be handled by the main game loop. Instead, the program's main function must call one function for handling the splash screen, and then a second function for handling the game itself. The latter function must contain the main game loop.

Time permitting, you may also wish to display a background image, high score list, etc. on this screen. But, these are not core features of the project.

Guidance: Mouse Input and Pointer Plotting

If the IKBD ISR from the previous section is fully and correctly implemented, the ISR should already be reading (and discarding) mouse packets. So that the program can keep track of the mouse's absolute screen location, maintain global x and y integer variables. Within the ISR add the Δx and Δy from each mouse packet to the global x and y variables. Similarly, a global variable can be used for maintaining mouse button state as obtained from each mouse button's header byte. The splash screen handling function, after plotting the menu, can continuously poll the button state. At each down-click, the mouse position can be consulted to determine

which menu option was selected (if any).

The main program **MUST NOT** access the mouse's absolute screen location via the global `x` and `y` variables. These should be accessed via functions, which you will create in the *input* module. This is identical to accessing the keyboard input, the main program does not access the global keyboard buffer, but rather uses appropriate system calls. Your mouse device driver, ISR and system calls, must be independent of your game program. You should be able to include only your device driver into any other program.

Two approaches may be considered for plotting a solid (non-flickering or tearing) mouse pointer. The first is to use double-buffering, which may be overkill. The second is to single-buffer the splash screen, and to very quickly plot the mouse pointer inside the VBL ISR¹².

When the mouse pointer has moved, it will be necessary to un-plot at the former location and to re-plot at the new location. One simple approach is to perform an XOR operation for both plotting and un-plotting. Alternatively, a “back fill” bitmap, of the same dimensions as the mouse pointer bitmap, can be maintained. It is used to hold a copy of the frame buffer region currently obscured by the pointer. When the pointer moves on, the uncovered region can be restored. Of course, a newly obscured region must be saved for later back filling.

Other Requirements

Your code must be highly readable and self-documenting, including proper indentation and spacing, descriptive variable and function names, etc. No one function should be longer than approximately 25-30 lines of code – decompose as necessary. Code which is unnecessarily complex or hard to read will be severely penalized.

For each function you develop, write a short header block comment which specifies:

1. Its purpose, from the caller's perspective (if not perfectly clear from the name);
2. The purpose of each input parameter (if not perfectly clear from the name);
3. The purpose of each output parameter and return value (if not perfectly clear from the name);
4. Any assumptions, limitations or known bugs.

At the top of each source file, write a short block comment which summarizes the common purpose of the data structures and functions in the file. This should also include a normal ID header, but should also include team member names and the author's name.

You should add inline comments if you need to explain an algorithm or clarify a particularly tricky block of code, but keep this to a minimum.

¹²Remember, the VBL ISR runs in response to a vertical blank – the brief period of time between screen refreshes. If the plotting operation is completed quickly enough, the entire mouse pointer bitmap will have been plotted before the next screen refresh begins.